

Modul 1: MySQL – Data Manipulation Language (DML) - INSERT

Topik

Penggunaan SQL Statement INSERT pada DBMS MySQL.

Tujuan

Setelah mempelajari modul ini, mahasiswa diharapkan dapat:

1. Memahami penggunaan SQL statement INSERT.
2. Menambahkan data ke dalam tabel menggunakan berbagai format INSERT.

Pendahuluan

DML merupakan istilah untuk beberapa sintaksis (syntax) dari SQL yang digunakan untuk melakukan perubahan pada data (isi tabel-tabel) dalam suatu database. DML terdiri dari 3 klausa utama yaitu:

1. **INSERT** : Menambah baris baru pada sebuah tabel
2. **UPDATE** : Mengubah nilai suatu baris pada sebuah tabel.
3. **DELETE** : Menghapus suatu baris dari sebuah tabel.

Pada DML terdapat dua jenis bahasa, yaitu :

1. High-Level(Non_procedural) DML.

- Digunakan secara interaktif (interpreter)
 - Dapat dijadikan satu dengan general purpose programming language (embedded)
- High-Level DML yang biasa digunakan secara interaktif disebut "Query Language".

2. Low-Level(Proedural) DML.

Digunakan secara embedded dalam suatu general purpose programming language

Bilamana kedua jenis DML diatas digunakan secara "embedded", maka : bahasa pemrograman yang digunakan disebut sebagai "Host Language" dan "DML-nya disebut "Sub Language"

Operasi INSERT

Operasi insert bertujuan untuk menyisipkan satu tuple baru ke dalam suatu relasi R.

Klausa pembentuk:

1. **INSERT**
2. **INTO**
3. **VALUES**

Format:

1. **INSERT INTO** nama_tabel (kolom1, kolom2, ...dst.) **VALUES** (nilai_kolom1, nilai_kolom2, ...dst.);
2. **INSERT INTO** nama_tabel **VALUES** (nilai_kolom1, nilai_kolom2, ...dst.); 3. [Salah satu dari kedua format sebelumnya], (nilai_kolom_kolom_baris1), (nilai_kolom_kolom_baris2), ...dst.

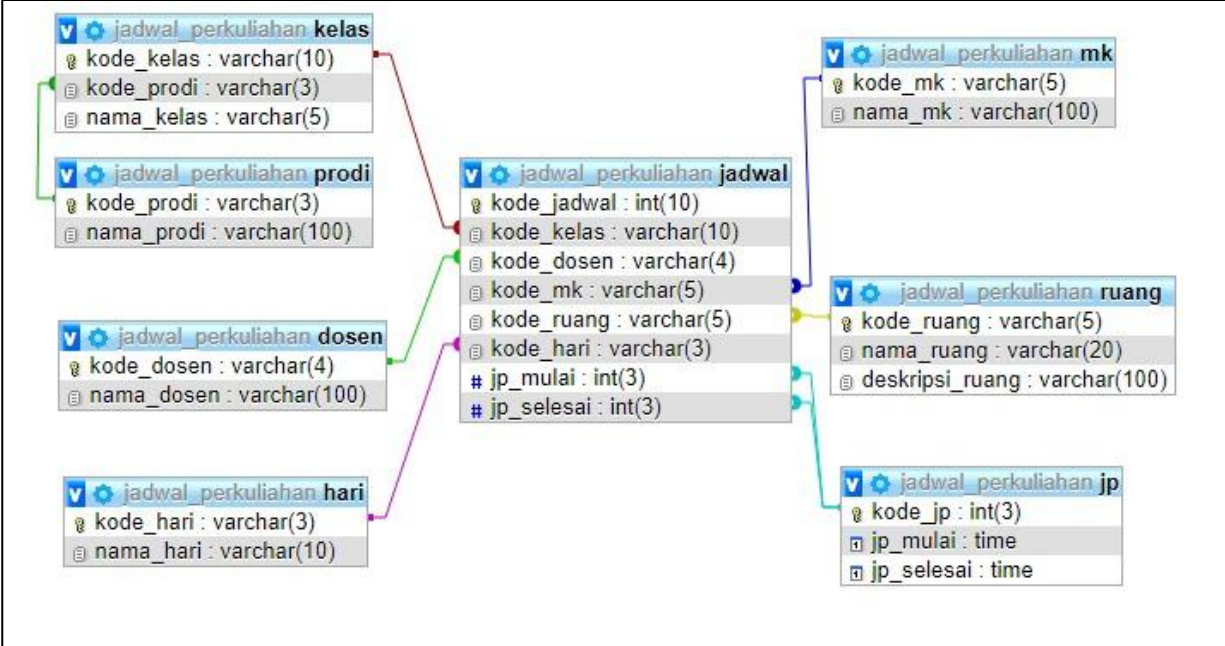
Operasi ini memungkinkan untuk melanggar empat jenis constraint sebagaimana dijelaskan berikut ini :

1. **DOMAIN Constraint** dapat dilanggar jika suatu nilai attribute yang diberikan tidak ada dalam domain yang berkorespondensi dengan attribute tadi.
2. **KEY Constraint** dapat dilanggar jika nilai key dalam tuple baru t sudah ada dalam tuple lain dalam relasi $r(R)$.
3. **ENTITY INTEGRITY** Constraint dapat dilanggar jika primary key dari tuple baru t adalah NULL
4. **REFERENTIAL INTEGRITY** Constraint dapat dilanggar jika nilai dari suatu foreign key dalam t mengacu ke suatu tuple yang tidak ada dalam relasi yang diacu.

Ada dua pilihan tindakan yang dapat dilakukan jika ada satu atau lebih constraint yang dilanggar akibat operasi insert, yaitu :

1. Menolak (reject) operasi insertion. Biasanya DBMS memberikan penjelasan mengapa proses insertion ditolak.
2. Berusaha memperbaiki alasan penolakan proses insertion. Dimana insertion akan diterima jika user melakukan perubahan nilai-nilai attribute sehingga insertion diterima.

Praktikum – Bagian 1: Percobaan Statement INSERT

Langkah	Keterangan
	Perhatikan skema/model relasional/EER diagram dari database berikut.  <pre> graph LR subgraph "jadwal_perkuliahan_kelas" k_kode_kelas[kode_kelas : varchar(10)] k_kode_prodi[kode_prodi : varchar(3)] k_nama_kelas[nama_kelas : varchar(5)] end subgraph "jadwal_perkuliahan_prodi" p_kode_prodi[kode_prodi : varchar(3)] p_nama_prodi[nama_prodi : varchar(100)] end subgraph "jadwal_perkuliahan_dosen" d_kode_dosen[kode_dosen : varchar(4)] d_nama_dosen[nama_dosen : varchar(100)] end subgraph "jadwal_perkuliahan_hari" h_kode_hari[kode_hari : varchar(3)] h_nama_hari[nama_hari : varchar(10)] end subgraph "jadwal_perkuliahan_jadwal" j_kode_jadwal[kode_jadwal : int(10)] j_kode_kelas[kode_kelas : varchar(10)] j_kode_dosen[kode_dosen : varchar(4)] j_kode_mk[kode_mk : varchar(5)] j_kode_ruang[kode_ruang : varchar(5)] j_kode_hari[kode_hari : varchar(3)] j_jp_mulai[jp_mulai : int(3)] j_jp_selesai[jp_selesai : int(3)] end subgraph "jadwal_perkuliahan_mk" mk_kode_mk[kode_mk : varchar(5)] mk_nama_mk[nama_mk : varchar(100)] end subgraph "jadwal_perkuliahan_ruang" r_kode_ruang[kode_ruang : varchar(5)] r_nama_ruang[nama_ruang : varchar(20)] r_deskripsi_ruang[deskripsi_ruang : varchar(100)] end subgraph "jadwal_perkuliahan_jp" jp_kode_jp[kode_jp : int(3)] jp_jp_mulai[jp_mulai : time] jp_jp_selesai[jp_selesai : time] end k_kode_prodi --> p_kode_prodi p_kode_prodi --> j_kode_prodi d_kode_dosen --> j_kode_dosen h_kode_hari --> j_kode_hari j_kode_kelas --> k_kode_kelas j_kode_mk --> mk_kode_mk j_kode_ruang --> r_kode_ruang j_kode_jp --> jp_kode_jp </pre>
1.	Untuk menambahkan data (mengisi) suatu tabel, digunakan statement (pernyataan) INSERT. Eksekusi SQL berikut untuk menambahkan 1 baris (record) baru pada tabel mk. <pre>INSERT INTO mk (kode_mk, nama_mk) VALUES ('02010', 'Basis Data');</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang dinyatakan di dalam tanda kurung () pertama. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. Pembahasan lebih lengkap mengenai SELECT dijadwalkan untuk disampaikan pada pertemuan berikutnya, namun secara umum, statement SELECT digunakan untuk menyajikan record yang ada pada suatu tabel. Karakter * akan menampilkan isi dari semua kolom yang ada pada tabel. <pre>SELECT * FROM mk</pre>

2.	Apabila data di-insert-kan pada semua kolom tabel, maka kita dapat langsung menggunakan klausa VALUES tanpa harus menuliskan nama-nama kolom dahulu. <pre>INSERT INTO mk VALUES('02041', 'Teknologi Data');</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk tanpa menyebutkan nama kolomnya. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
3.	Untuk menambahkan beberapa kolom sekaligus dalam 1 statement digunakan statement dengan format seperti berikut. <pre>INSERT INTO mk VALUES (`02004`, `Aljabar Linier`), (`02005`, `Analisis Dan Desain Berorientasi Objek`), (`02006`, `Bahasa Indonesia`);</pre> Statement SQL tersebut menambahkan 3 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk tanpa menyebutkan nama kolomnya. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
4.	Dan seperti berikut, jika hanya kolom tertentu saja yang akan diberi nilai dengan cara menyebutkan nama kolomnya. <pre>INSERT INTO mk (kode_mk, nama_mk) VALUES (`02001`, `Agama`), (`02002`, `Aljabar Linier`), (`02003`, `Algoritma dan Struktur Data`);</pre>

	Statement SQL tersebut menambahkan 3 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
5.	Statement INSERT juga dapat dieksekusi dengan menggunakan klausa SET alih-alih VALUES. <pre>INSERT INTO mk VALUES kode_mk = `02012`, nama_mk = `Digital Entrepreneurship`;</pre> Statement SQL tersebut menambahkan 1 baris baru ke tabel mk pada kolom yang ada pada struktur tabel mk. Untuk melihat hasil SQL yang kita eksekusi tersebut, gunakan statement SELECT seperti berikut. <pre>SELECT * FROM mk</pre>
6.	Pada statement INSERT juga dapat digunakan klausa SELECT. Misalnya kita ingin menyalin semua baris pada tabel mk ke tabel mk_backup, maka kita dapat menggunakan SQL berikut. (Buat terlebih dahulu tabel “mk_backup” dengan struktur tabel yang sama dengan tabel “mk”) <pre>CREATE TABLE mk_backup (kode_mk VARCHAR(5), nama_mk VARCHAR(100));</pre>
7.	Kemudian masukkan data dari tabel mk ke tabel mk_backup <pre>INSERT INTO mk_backup SELECT * FROM mk;</pre>
8.	Setelah berhasil mengeksekusi SQL tersebut, lanjutkan ke SubTopik selanjutnya.

Modul 2: MySQL – Data Manipulation Language (DML) – UPDATE

Topik

Penggunaan SQL Statement UPDATE pada DBMS MySQL.

Tujuan

Setelah mempelajari modul ini, mahasiswa diharapkan dapat:

1. Memahami penggunaan SQL statement UPDATE.
2. Mengubah nilai satu atau lebih baris pada sebuah tabel menggunakan berbagai format UPDATE.

Pendahuluan

Operasi UPDATE digunakan untuk mengubah nilai satu atau lebih atribut dalam satu atau lebih tuple (baris) pada suatu tabel. Operasi ini hanya dapat melanggar referential integrity jika nilai yang diubah adalah foreign key yang mengacu pada tuple lain dalam basis data.

Operasi UPDATE

Operasi update digunakan untuk merubah nilai-nilai satu atau lebih attribute dalam satu atau lebih tuple dalam sejumlah relasi R.

Klausa pembentuk:

1. **UPDATE**
2. **SET**
3. **WHERE** [opsional]

Format:

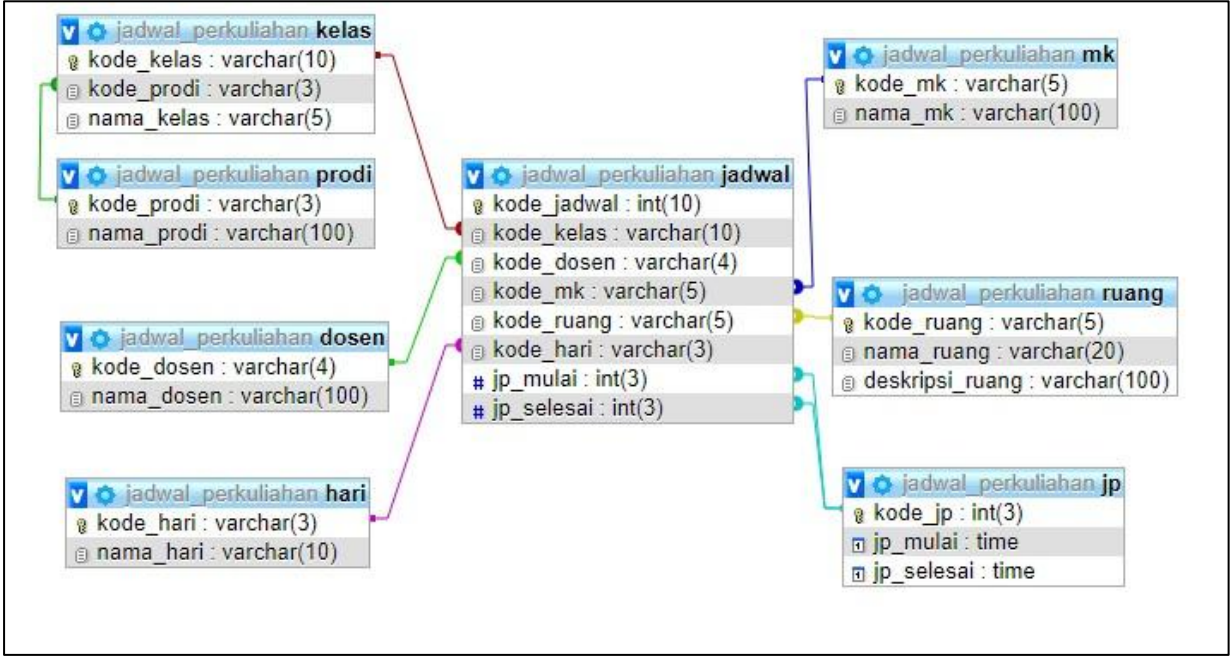
1. **UPDATE** nama_tabel **SET** nama_kolom = nilai_baru **WHERE** nama_kolom_patokan [**operator_perbandingan**] nilai_patokan;
2. **UPDATE** nama_tabel **SET** nama_kolom1 = nilai_baru1, nama_kolom2 = nilai_baru2, ...dst. **WHERE** nama_kolom_patokan [**operator_perbandingan**] nilai_patokan;

*Operator perbandingan/comparison operator dapat berupa: =, <, >, <=, >=, <>

Operasi UPDATE bisa dilakukan pada tiga jenis attribut, dengan permasalahan yang berbeda sebagaimana berikut ini :

1. Modifikasi nilai suatu foreign key, maka DBMS harus melakukan pengecekan bahwa nilai-nilai baru yang diberikan mengacu pada tuple yang ada dalam relasi-relasi yang dijadikan acuan.
2. modifikasi nilai suatu primary key serupa dengan proses deletion satu tuple dan inserting yang lain pada tempat yang sama. Akibatnya, pilihan- pilihan seperti yang dilakukan pada operasi INSERT dan DELETE dapat dipakai agar modifikasi tidak melanggar constraint.
3. modifikasi suatu attribute yang bukan primary key atau bukan foreign key biasanya tidak akan menimbulkan masalah. DBMS hanya perlu untuk mengecek apakah nilai-nilai baru yang diberikan mempunyai tipe data dan domain yang valid.

Praktikum - Bagian 2: Percobaan Statement UPDATE

Langkah	Keterangan
	Perhatikan skema/model relasional/EER diagram dari database berikut. 
1	UPDATE digunakan untuk mengubah nilai pada satu atau beberapa baris dalam sebuah tabel. Cek terlebih dahulu dan pastikan data yang diperlukan sudah tersimpan dalam tabel jadwal sebelum melanjutkan ke praktikum bagian 3 pada kolom Try Query Data dibawah kolom jawaban pada browser dengan menggunakan perintah SELECT. Format dasar statement Update ini adalah sebagai berikut: <pre>UPDATE jadwal SET kode_dosen='D010'</pre>

	Statement tersebut mengubah nilai SEMUA baris dari tabel jadwal pada kolom kode_dosen dengan nilai D010. Apabila kita tampilkan isi tabel, maka sekarang semua mata kuliah akan diampu oleh dosen dengan kode_dosen tersebut. Tampilkan isi data menggunakan statement SELECT berikut <pre>SELECT * FROM jadwal</pre>
2	Untuk mengubah nilai pada baris tertentu saja, kita tambahkan klausa WHERE pada statement UPDATE. Misalkan kita akan menjadikan dosen dengan kode_dosen D022 sebagai pengampu mata kuliah dengan kode_mk 02010, maka dapat digunakan SQL sebagai berikut: <pre>UPDATE jadwal SET kode_dosen = `D022`, WHERE kode_mk = `02017`;</pre> <pre>SELECT * FROM jadwal WHERE kode_mk = `02017`;</pre>
3	Klausa WHERE tidak selalu hanya membatasi UPDATE pada 1 baris saja, ia juga bisa memberlakukan UPDATE pada banyak baris sekaligus. Semuanya tergantung pada kondisi yang kita tentukan. Statement berikut ini akan mengosongkan kode_dosen untuk semua mata kuliah yang diampu oleh dosen dengan kode_dosen D010. <pre>UPDATE jadwal SET kode_dosen = NULL WHERE kode_dosen = 'D010'</pre> <pre>SELECT * FROM jadwal</pre>
4	Untuk mengubah beberapa kolom sekaligus dalam satu kali eksekusi statement UPDATE, dapat digunakan format berikut. <pre>UPDATE jadwal SET kode_dosen = 'D012', kode_ruang = '0702' WHERE kode_kelas = '2021020204'</pre>

	<pre>SELECT * FROM jadwal WHERE kode_kelas = '2021020204'</pre>
5	Kita juga dapat menggunakan statement UPDATE dengan SELECT. Misalkan kita ingin mengeset kode_dosen dari kode_mk '02010' dengan kode_dosen dari dosen yang bernama 'Dika Rizky Yuniarto SKom., MKom.', maka dapat digunakan SQL dengan format berikut. <pre>UPDATE jadwal SET kode_dosen = (SELECT kode_dosen FROM dosen WHERE nama_dosen = 'Dika Rizky Yuniarto Skom., MKom.') WHERE kode_mk = '02010'</pre> <pre>SELECT * FROM jadwal WHERE kode_mk = `02017`;</pre>
6	Setelah berhasil mengeksekusi SQL tersebut, lanjutkan ke <u>Praktikum - Bagian 3</u>.

Modul 3: MySQL – Data Manipulation Language (DML) – DELETE

Topik

Penggunaan SQL Statement DELETE pada DBMS MySQL.

Tujuan

Setelah mempelajari modul ini, mahasiswa diharapkan dapat:

1. Memahami penggunaan SQL statement DELETE.
2. Menghapus satu atau lebih baris dari sebuah tabel menggunakan berbagai format DELETE.

Pendahuluan

Operasi DELETE digunakan untuk menghapus satu atau beberapa tuple (baris) di dalam suatu tabel. Operasi ini hanya dapat melanggar referential integrity jika tuple yang dihapus diacu oleh kunci tamu (foreign key) dari tuple lain dalam basis data.

Operasi DELETE

Operasi delete bertujuan untuk menghapus satu atau beberapa tuple di dalam suatu relasi R. Operasi ini hanya dapat melanggar **referential integrity**, jika tuple yang dihapus diacu oleh kunci-kunci tamu dari tuple yang lain dalam basis data.

Klausa pembentuk:

1. **DELETE**
2. **FROM**
3. **WHERE** [opsional]

Format:

1. **DELETE FROM** nama_tabel **WHERE** nama_kolom_patokan [**operator_perbandingan**] nilai_patokan;
2. **DELETE * FROM** nama_tabel; atau **DELETE FROM** nama_tabel;

*Operator perbandingan/comparison operator dapat berupa: =, <, >, <=, >=, <>

Ada empat pilihan tindakan yang dapat dilakukan jika suatu deletion melanggar constraint yang telah ditentukan, yaitu :

1. Menolak(reject) proses deletion.
2. Berusaha untuk melakukan “cascade deletion”, yaitu dengan menghapus sejumlah tuple yang mengacu pada tuple yang akan dihapus.
3. Melakukan modifikasi nilai attribute yang mengacu pada tuple yang dihapus, yaitu setiap nilai diset NULL atau diganti dengan nilai dari tuple lain yang valid sebagai acuan baru. Akan tetapi, bila attribute yang mengacu yang menyebabkan pelanggaran adalah bagian dari primary key, maka ia tidak dapat diset NULL (karena melanggar entity integrity).
4. Kombinasi 2 dan 3.

Practicum - Part 3: DELETE Statement Experiment

Step	Description
1	DELETE digunakan untuk menghapus satu atau beberapa baris dari sebuah tabel. Misalkan kita ingin menghapus jadwal yang memiliki nilai pada kode_dosen , maka format dasar pernyataan sebagai berikut dapat kita gunakan:
	<pre>DELETE FROM jadwal WHERE kode_dosen IS NOT NULL;</pre>
	<pre>SELECT * FROM jadwal</pre>
2	HATI-HATI ketika kita menggunakan pernyataan DELETE tanpa WHERE! Coba jalankan sintaks SQL berikut ini:
	<pre>DELETE FROM jadwal;</pre>
	Semua data dalam satu tabel jadwal akan hilang!
	<pre>SELECT * FROM jadwal</pre>

```
UPDATE jadwal
  SET kode_dosen = (SELECT kode_dosen FROM dosen
    WHERE nama_dosen='Dika Rizky Yudianto SKom., MKom.')
  WHERE kode_mk = '02010'
```

```
INSERT INTO mk VALUES
  kode_mk = `02012`,
  nama_mk = `Digital Entrepreneurship`;
```